

Guide de Preparation

Entretien Technique Java / Spring Boot

Software Engineer Junior (0–2 ans d'experience)

Environ 150 questions-reponses classees par theme

Java Core – Spring Core – Spring Boot – REST API – JPA/Hibernate
SQL – Spring Security – Maven – Git – Docker – Tests – Design Patterns

A utiliser comme support de revision active : cachez la reponse et essayez de la reformuler a voix haute avant de la lire.

Fiche de preparation personnelle

Table des matières

1	Java Core	2
1.1	Variables et types	2
1.2	Programmation orientee objet	3
1.3	Collections	4
1.4	Exceptions	5
1.5	Java 8+ (Lambdas, Stream API, Optional)	6
1.6	Multithreading	7
2	Spring Core	9
3	Spring Boot	12
4	Developpement Web et REST API	15
5	Spring Data JPA / Hibernate	17
6	SQL	20
7	Spring Security et JWT	22
8	Tests (JUnit, Mockito)	23
9	Maven, Git, Docker	25
9.1	Maven	25
9.2	Git	25
9.3	Docker	26
10	Design Patterns	27
11	Questions sur votre projet	29
12	Pieges classiques et questions de reflexion	31
13	Exercices de code en direct	32

1. Java Core

1.1 Variables et types

Q.

Quelle est la difference entre un type primitif et un objet en Java ?

Reponse attendue

Un type primitif (`int`, `boolean`, `double`, `char`, ...) stocke directement sa valeur sur la pile (stack) et n'est pas nullable. Un objet (`Integer`, `String`, ...) est une reference vers une zone memoire sur le tas (heap) ; il peut valoir `null`, possede des methodes, et transite par les wrapper classes pour l'autoboxing/unboxing avec les collections generiques.

Q.

Pourquoi la classe `String` est-elle immuable en Java ?

Reponse attendue

Trois raisons principales : (1) securite – une `String` passee en parametre ne peut pas etre modifiee a l'insu de l'appelant (utile pour les mots de passe, les noms de classe en reflexion) ; (2) le *String Pool* peut reutiliser en toute securite la meme instance pour deux litteraux identiques, economisant de la memoire ; (3) thread-safety native, sans synchronisation, puisqu'un objet immuable ne peut jamais etre dans un etat incoherent partage entre threads.

Q.

`StringBuilder` vs `StringBuffer` : quelle difference ?

Reponse attendue

Les deux sont mutables et servent a concatener efficacement des chaines sans recreer un nouvel objet a chaque `+=`. `StringBuffer` est **synchronized** (thread-safe mais plus lent), tandis que `StringBuilder` ne l'est pas (plus rapide, a utiliser des qu'on est dans un contexte mono-thread, ce qui est le cas la grande majorite du temps).

Q.

Quelle est la difference entre `==` et `.equals()` ?

Reponse attendue

`==` compare des references (l'adresse memoire) pour les objets, ou la valeur brute pour les primitifs. `.equals()` est une methode qui, par default (dans `Object`), fait la meme chose que `==`, mais qui est souvent redefinie (ex : dans `String`, `Integer`) pour comparer le contenu logique des objets.

Piege classique

`new String("abc") == "abc"` renvoie `false` (deux objets differents), alors que `"abc" == "abc"` renvoie `true` grace au String Pool. C'est un exemple classique en entretien pour verifier

qu'on comprend la difference reference/valeur.

Q.

A quoi sert `hashCode()` et quel est son lien avec `equals()` ?

Reponse attendue

`hashCode()` retourne un entier utilise par les structures de type table de hachage (`HashMap`, `HashSet`) pour placer rapidement un objet dans un "bucket". Le contrat impose : si `a.equals(b)` est vrai, alors `a.hashCode() == b.hashCode()` doit etre vrai. L'inverse n'est pas obligatoire (deux objets differents peuvent avoir le meme hash : c'est une *collision*). Si on redefinit `equals()` sans redefinir `hashCode()`, on casse ce contrat et les collections a base de hash deviennent incoherentes.

1.2 Programmation orientee objet

Q.

Quels sont les 4 principes fondamentaux de la POO ?

Reponse attendue

Encapsulation (cacher l'etat interne derriere des methodes publiques), **heritage** (reutiliser/etendre un comportement via `extends`), **polymorphisme** (une meme methode se comporte differemment selon le type reel de l'objet, via `@Override`) et **abstraction** (exposer un contrat sans exposer les details d'implementation, via interfaces/classes abstraites).

Q.

Heritage vs composition : quand choisir l'un ou l'autre ?

Reponse attendue

L'heritage exprime une relation "est-un" (*is-a*) et cree un couplage fort avec la classe parente (fragile base class problem). La composition exprime une relation "a-un" (*has-a*) : on injecte un objet collaborateur au lieu d'en heriter, ce qui est plus flexible et plus facile a tester. Regle generale largement acceptee : "*Favor composition over inheritance*" – Spring lui-meme repose massivement sur la composition via l'injection de dependances.

Q.

Interface vs classe abstraite : quelle difference et quand utiliser chacune ?

Reponse attendue

Une interface definit un contrat pur (avant Java 8, aucune implementation ; depuis Java 8, elle peut avoir des methodes `default/static`) et une classe peut implementer plusieurs interfaces. Une classe abstraite peut avoir un etat (champs), des constructeurs et du code partage, mais une classe ne peut en etendre qu'une seule. On utilise une interface pour definir une capacite (`Comparable`, `Runnable`) et une classe abstraite quand plusieurs sous-classes partagent une implementation commune non triviale.

Q.

Qu'est-ce que le polymorphisme et comment se manifeste-t-il en Java ?

Reponse attendue

C'est la capacite d'appeler une methode sur une reference de type parent/interface et de voir s'exécuter l'implementation de la classe reelle de l'objet (*polymorphisme dynamique*, resolu a l'exécution via la table des methodes virtuelles). Spring l'exploite constamment : on programme contre des interfaces (`List`, `Repository`) et le conteneur injecte l'implementation concrete.

1.3 Collections

Q.

`ArrayList` vs `LinkedList` : quelle difference et quand utiliser chacune ?

Reponse attendue

`ArrayList` est adosse a un tableau redimensionnable : acces par indice en $O(1)$, mais insertion/suppression au milieu couteuse en $O(n)$ (decalage des elements). `LinkedList` est une liste doublement chainee : insertion/suppression en $O(1)$ si on a deja le noeud (ex : en tete/queue), mais acces par indice en $O(n)$. En pratique, `ArrayList` est le choix par default dans 90% des cas car les acces sequentiels/aleatoires dominant.

Q.

Comment fonctionne une `HashMap` en interne ?

Reponse attendue

Une `HashMap` stocke un tableau de "buckets". Pour inserer une paire cle/valeur, on calcule `hashCode()` de la cle, on applique un spreading de bits pour reduire les collisions, puis on prend le modulo de la taille du tableau pour trouver le bucket. Depuis Java 8, si un bucket contient plus de 8 elements, la liste chainee interne se transforme en arbre rouge-noir pour garder une recherche en $O(\log n)$ au lieu de $O(n)$ dans le pire cas.

Q.

Pourquoi dit-on que `HashMap` offre du $O(1)$ en moyenne ?

Reponse attendue

Parce qu'en l'absence de collisions, trouver le bon bucket via le hash puis acceder a l'element est un temps constant, independant du nombre d'elements stockes. C'est une complexite *amortie moyenne* : dans le pire cas (toutes les cles dans le meme bucket), cela degene en $O(\log n)$ (arbre) ou $O(n)$ (liste chainee avant Java 8).

Q.

Que se passe-t-il si deux cles differentes ont le meme `hashCode()` ?

Reponse attendue

C'est une *collision* : les deux entrees atterrissent dans le meme bucket. La `HashMap` les stocke toutes les deux dans ce bucket (liste chaine ou arbre) et utilise `equals()` pour les distinguer lors d'une recherche – elle parcourt le bucket et compare chaque cle avec `equals()` jusqu'a trouver la bonne.

Q.

`HashSet` vs `TreeSet` : quelle difference ?

Reponse attendue

`HashSet` est adosse a une `HashMap` : pas d'ordre garanti, insertion/recherche en $O(1)$ moyen. `TreeSet` est adosse a un arbre rouge-noir (`TreeMap`) : les elements sont toujours tries (ordre naturel ou via `Comparator`), mais les operations sont en $O(\log n)$.

Q.

A quoi sert `ConcurrentHashMap` et en quoi differe-t-elle d'une `HashMap` synchronisee ?

Reponse attendue

`ConcurrentHashMap` permet des lectures/ecritures concurrentes sans bloquer toute la map : elle segmente les verrous au niveau des buckets (au lieu d'un verrou global comme `Collections.synchronizedMap()`), ce qui offre un bien meilleur debit en environnement multi-thread. Elle ne leve jamais de `ConcurrentModificationException` lors d'une iteration concurrente avec une modification.

1.4 Exceptions

Q.

Quelle est la difference entre *checked* et *unchecked exception* ?

Reponse attendue

Une *checked exception* (heritant de `Exception` mais pas de `RuntimeException`, ex : `IOException`) doit obligatoirement etre geree (try/catch) ou declaree (`throws`) a la compilation – elle represente une condition anticipable et recuperable. Une *unchecked exception* (heritant de `RuntimeException`, ex : `NullPointerException`) n'a pas besoin d'etre declaree – elle signale generalement un bug de programmation. Spring, par convention, ne leve que des unchecked exceptions dans son ecosysteme (ex : `DataAccessException`).

Q.

A quoi sert le `try-with-resources` ?

Reponse attendue

Il garantit la fermeture automatique de toute ressource implementant `AutoCloseable` (flux, connexion JDBC, ...) a la sortie du bloc `try`, meme en cas d'exception, sans avoir a ecrire de bloc `finally` manuel avec un `close()` risquant lui-meme de lever une exception.

Q.

Le bloc `finally` est-il toujours execute ?

Reponse attendue

Presque toujours – y compris si un `return` est present dans le `try` ou le `catch`. Les seules exceptions sont : un appel a `System.exit()`, un crash de la JVM, ou un thread tue de force. Piege classique : si `finally` contient lui-meme un `return`, il ecrase silencieusement la valeur de retour ou l'exception du `try/catch`.

1.5 Java 8+ (Lambdas, Stream API, Optional)

Q.

Qu'est-ce qu'une expression lambda et a quoi sert-elle ?

Reponse attendue

C'est une syntaxe concise pour implementer une interface fonctionnelle (une interface avec une seule methode abstraite, ex : `Runnable`, `Comparator`, `Function<T,R>`) sans classe anonyme. Elle rend le code fonctionnel (Stream API, callbacks) beaucoup plus lisible.

```
1 List<String> noms = users.stream()  
2   .filter(u -> u.getAge() > 18)  
3   .map(User::getName)  
4   .sorted()  
5   .collect(Collectors.toList());
```

Q.

Quelle est la difference entre `map()` et `flatMap()` dans la Stream API ?

Reponse attendue

`map()` transforme chaque element en un autre element (relation 1-vers-1), par exemple `String -> Integer`. `flatMap()` transforme chaque element en un *Stream*, puis aplatit tous ces streams en un seul (relation 1-vers-plusieurs) : utile quand on a une liste de listes et qu'on veut obtenir une seule liste "aplatie".

Q.

A quoi servent `filter()` et `reduce()` ?

Reponse attendue

`filter()` garde uniquement les elements verifiant un predicat (`Predicate<T>`). `reduce()` combine tous les elements d'un stream en une seule valeur en appliquant successivement une fonction binaire (ex : sommer une liste d'entiers : `stream.reduce(0, Integer::sum)`).

Q.

Pourquoi utiliser `Optional` plutot que de retourner `null` ?

Reponse attendue

`Optional<T>` rend explicite, au niveau du type de retour, qu'une valeur peut être absente – l'appelant est donc obligé de gérer ce cas (`isPresent()`, `orElse()`, `map()`) au lieu de risquer une `NullPointerException` silencieuse plus loin dans le code. C'est un outil de *design* pour les valeurs de retour, pas pour les champs d'entité ou les paramètres de méthode (à éviter dans ces cas selon les conventions Java).

1.6 Multithreading

Q.

Thread vs Runnable vs Callable : quelle différence ?

Reponse attendue

`Thread` est la classe représentant un fil d'exécution réel. `Runnable` est une interface fonctionnelle (`void run()`) qui décrit une tâche sans valeur de retour ni exception checked – on la passe à un `Thread` ou à un `ExecutorService`. `Callable<V>` est similaire mais sa méthode `call()` retourne une valeur et peut lever une exception checked, ce qui la rend compatible avec un `Future<V>`.

Q.

Qu'est-ce qu'un `ExecutorService` et pourquoi l'utiliser plutôt que de créer des `Thread` manuellement ?

Reponse attendue

C'est un pool de threads réutilisables géré par la JVM : on lui soumet des tâches (`Runnable/Callable`) et il les distribue aux threads disponibles. Cela évite le coût de création/destruction répétée de threads et permet de limiter le parallélisme (éviter de saturer le système). Un `Future<V>` récupère le résultat asynchrone d'une tâche soumise.

Q.

Que fait le mot-clé `synchronized` ?

Reponse attendue

Il garantit qu'un seul thread à la fois peut exécuter le bloc/méthode protégé pour un moniteur (objet verrou) donné, empêchant les accès concurrents non contrôlés à une ressource partagée. C'est le mécanisme de base pour éviter les *race conditions*, au prix d'un risque de contention/latence si trop de threads attendent le même verrou.

Q.

Qu'est-ce qu'un *deadlock* et qu'est-ce qu'une *race condition* ?

Reponse attendue

Une **race condition** survient quand le résultat d'un programme dépend de l'ordre d'exécution non déterministe de threads concurrents accédant à un état partagé sans synchronisation.

correcte. Un **deadlock** survient quand deux threads (ou plus) s'attendent mutuellement indéfiniment, chacun detenant une ressource dont l'autre a besoin (ex : Thread A tient le verrou 1 et attend le verrou 2, pendant que Thread B tient le verrou 2 et attend le verrou 1).

2. Spring Core

Q.

Qu'est-ce que Spring Framework, en une phrase ?

Reponse attendue

Un framework Java qui fournit un conteneur d'injection de dependances (IoC container) et un ecosysteme de modules (MVC, Data, Security, ...) permettant de construire des applications faiblement couplees, testables et modulaires sans gerer soi-meme le cycle de vie des objets.

Q.

Qu'est-ce que l'Inversion de Controle (IoC) ?

Reponse attendue

C'est un principe de conception ou le *contrôle de la creation et du cablage des objets* est delegue a un framework (le conteneur Spring) plutot que d'être fait manuellement par le code applicatif avec `new`. Au lieu que la classe A aille elle-meme chercher/creer son dependance B, c'est le conteneur qui lui "fournit" B : le controle du flux est *inverse*.

Q.

Qu'est-ce que l'Injection de Dependances (DI) et quels sont ses 3 types ?

Reponse attendue

La DI est la technique concrete par laquelle l'IoC est realisee : le conteneur `ApplicationContext` instancie les beans et leur injecte leurs dependances. Les 3 types sont : **injection par constructeur** (dependances passees en parametres du constructeur), **injection par setter** (via une methode `setXxx()` annotee `@Autowired`), et **injection par champ** (`@Autowired` directement sur l'attribut, via reflexion).

A retenir

L'**injection par constructeur** est recommandee car : (1) elle permet de declarer les champs `final` (immutabilite, thread-safety); (2) elle rend les dependances obligatoires explicites et empeche de construire un objet dans un etat incomplet; (3) elle facilite les tests unitaires purs (on peut instancier la classe avec `new` et des mocks, sans conteneur Spring); (4) elle expose immediatement un probleme de conception si le constructeur prend trop de parametres (trop de responsabilites).

```
1 @Service
2 public class OrderService {
3     private final PaymentClient paymentClient; // final -> immutable
4
5     // Depuis Spring 4.3, @Autowired est optionnel
6     // s'il n'y a qu'un seul constructeur
7     public OrderService(PaymentClient paymentClient) {
8         this.paymentClient = paymentClient;
9     }
10 }
```

Q.

Qu'est-ce qu'un Bean Spring et qui le cree ?

Reponse attendue

Un *bean* est simplement un objet Java dont le cycle de vie (instanciation, configuration, injection de dependances, destruction) est gere par le conteneur `ApplicationContext`, et non par le code applicatif. Il est cree soit via un scan de composants (`@Component` et derives), soit declare explicitement dans une classe `@Configuration` via une methode annotee `@Bean`.

Q.

Quels sont les principaux scopes de bean, et quelle est la difference entre *singleton* et *prototype* ?

Reponse attendue

Par default, un bean Spring est **singleton** : une seule instance est creee par conteneur et partagee partout ou elle est injectee. Le scope **prototype** cree une nouvelle instance a chaque injection/demande. D'autres scopes existent dans un contexte web : **request** (une instance par requete HTTP) et **session** (une instance par session utilisateur).

Q.

Pouvez-vous decire le cycle de vie d'un bean Spring ?

Reponse attendue

Schematiquement : (1) instanciation, (2) injection des dependances (constructeur/setter/champ), (3) appel des **Aware** interfaces si implemtees (ex : `ApplicationContextAware`), (4) appel de `@PostConstruct` (ou `InitializingBean.afterPropertiesSet()`), (5) le bean est pret a l'emploi, (6) a la fermeture du contexte, appel de `@PreDestroy` (ou `DisposableBean.destroy()`).

Q.

Quelle est la difference entre `@Component`, `@Service`, `@Repository` et `@Controller` ?

Reponse attendue

Techniquement, ce sont toutes des specialisations de `@Component` (le scanner de composants les traite de la meme facon), mais elles portent une intention semantique differente et certaines ajoutent un comportement : `@Repository` active la traduction automatique des exceptions techniques (JDBC, Hibernate) en `DataAccessException` Spring uniformes ; `@Service` marque la couche de logique metier ; `@Controller` marque un composant MVC qui gere les requetes web. Utiliser la bonne annotation ameliore la lisibilite de l'architecture en couches.

Q.

Quelle est la difference entre `@Configuration` + `@Bean` et `@Component` ?

Reponse attendue

@Component (et derives) marque une classe pour etre auto-detectee et instanciee par le scanner – on l'utilise pour son *propre* code. **@Bean**, place sur une methode d'une classe **@Configuration**, sert a declarer explicitement comme bean un objet qu'on ne peut pas annoter directement (ex : une classe d'une librairie tierce) ou dont la creation necessite de la logique personnalisee.

3. Spring Boot

Q.

Quelle est la difference entre Spring Framework et Spring Boot ?

Reponse attendue

Spring Boot ne remplace pas Spring Framework : il s'appuie dessus en ajoutant des *opinionated defaults* pour eliminer la configuration repetitive (plus besoin d'ecrire des kilometres de XML ou de configurer manuellement le `DispatcherServlet`). Il embarque un serveur (Tomcat/Jetty/Undertow) directement dans l'application (`jar` executable), fournit l'auto-configuration, et regroupe les dependances via des *starters*.

Q.

Que cache l'annotation `@SpringBootApplication` ?

Reponse attendue

C'est une annotation composite qui regroupe trois annotations : `@SpringBootConfiguration` (specialisation de `@Configuration`, indique que la classe peut declarer des beans), `@EnableAutoConfiguration` (declenche l'auto-configuration basee sur le classpath) et `@ComponentScan` (scanne automatiquement les composants du package courant et de ses sous-packages).

Q.

Comment fonctionne l'auto-configuration dans Spring Boot ?

Reponse attendue

Au demarrage, Spring Boot inspecte les JARs presents dans le classpath et les proprietes de configuration, puis active conditionnellement des classes de configuration pre-ecrites grace a des annotations comme `@ConditionalOnClass` (si telle classe est presente), `@ConditionalOnMissingBean` (si l'utilisateur n'a pas deja defini son propre bean), `@ConditionalOnProperty`. Par exemple, si le driver H2 est sur le classpath et qu'aucune `DataSource` n'est definie manuellement, Spring Boot en cree automatiquement une en memoire.

Piege classique

L'auto-configuration ne *remplace* jamais un bean explicitement defini par le developpeur : elle ne s'active que via `@ConditionalOnMissingBean`. Si un candidat pose une question du type "j'ai defini mon propre `DataSource` mais Spring Boot en cree quand meme un deuxieme", la reponse est presque toujours une erreur de configuration (mauvais package scanne, bean mal nomme), pas un bug de l'auto-configuration.

Q.

Qu'est-ce qu'un Spring Boot Starter ? Citez-en quelques-uns.

Reponse attendue

Un starter est un descripteur de dependances "tout-en-un" qui regroupe toutes les librairies necessaires a une fonctionnalite, avec des versions compatibles entre elles (gerees via le parent `spring-boot-starter-parent` ou le BOM). Exemples : `spring-boot-starter-web` (MVC + Tomcat embarque + Jackson), `spring-boot-starter-data-jpa` (Hibernate + Spring Data), `spring-boot-starter-security`, `spring-boot-starter-test` (JUnit + Mockito + AssertJ).

Q.

Quelle est la difference entre `application.properties` et `application.yml` ?

Reponse attendue

Les deux configurent l'application de la meme facon (memes cles, meme mecanisme de chargement) ; `.yml` utilise une syntaxe hierarchique par indentation, plus lisible pour les configurations imbriquees, alors que `.properties` utilise des cles a points (`server.port=8080`). On ne mixe generalement pas les deux dans un meme projet.

Q.

Comment injecter une valeur de configuration comme `server.port` dans une classe ?

Reponse attendue

Avec `@Value("$server.port")` sur un champ pour une valeur simple, ou en creant une classe annotee `@ConfigurationProperties(prefix = "...")` pour regrouper plusieurs proprietes liees dans un objet type-safe – approche recommandee des qu'on a plus de 2-3 proprietes liees.

Q.

A quoi servent les Profiles Spring (`@Profile`, `application-dev.yml`) ?

Reponse attendue

Ils permettent d'activer des beans ou des valeurs de configuration differentes selon l'environnement d'execution (dev, test, prod) sans dupliquer le code. On active un profil via la propriete `spring.profiles.active=dev` (ou une variable d'environnement), et Spring Boot charge alors `application.yml` puis le complete/surcharge avec `application-dev.yml`. En pratique, les secrets de prod (mots de passe, cles API) doivent venir de variables d'environnement plutot que d'etre commit dans un fichier de profil.

Q.

L'application met du temps a demarrer ou consomme trop de memoire : que verifiez-vous en premier ?

Reponse attendue

On active/consulte les endpoints **Spring Boot Actuator** : `/actuator/health` (etat global et des dependances comme la base de donnees), `/actuator/metrics` (JVM, pool de connexions, requetes HTTP), `/actuator/env` (proprietes effectivement chargees, utile pour detecter une mauvaise config). Cote demarrage, on peut aussi activer le mode debug de l'auto-configuration

(`-debug`) pour voir quelles configurations sont évaluées, ou profiler les beans les plus longs à instancier.

4. Developpement Web et REST API

Q.

Quelle est la difference entre `@Controller` et `@RestController` ?

Reponse attendue

`@RestController` = `@Controller` + `@ResponseBody`. Avec `@Controller` seul, le retour d'une methode est interprete comme le nom d'une vue a resoudre (JSP, Thymeleaf). Avec `@RestController`, la valeur retournee est serialisee directement dans le corps de la reponse HTTP (JSON par default via Jackson), ce qui en fait le choix standard pour une API REST.

Q.

Quand utiliser GET, POST, PUT, PATCH et DELETE ?

Reponse attendue

GET : lire une ressource, sans effet de bord (idempotent, cacheable). **POST** : creer une nouvelle ressource ou declencher une action non idempotente. **PUT** : remplacer entierement une ressource existante (idempotent : appeler 10 fois donne le meme etat final). **PATCH** : modifier partiellement une ressource. **DELETE** : supprimer une ressource (idempotent).

Q.

Citez les codes HTTP les plus utilises et leur signification.

Reponse attendue

200 OK (succes generique), **201 Created** (creation reussie, souvent avec un header Location), **204 No Content** (succes sans corps de reponse, ex : DELETE), **400 Bad Request** (requete invalide, ex : validation echouee), **401 Unauthorized** (authentification manquante/invalide), **403 Forbidden** (authentifie mais pas autorise), **404 Not Found** (ressource inexistante), **500 Internal Server Error** (erreur cote serveur non geree).

Q.

Quelle est la difference entre `@PathVariable` et `@RequestParam` ?

Reponse attendue

`@PathVariable` extrait une valeur directement depuis le chemin de l'URL (ex : `/users/{id}`), typiquement pour identifier une ressource precise. `@RequestParam` extrait un parametre de la query string (ex : `?page=2&size=10`), typiquement pour du filtrage, du tri ou de la pagination – des parametres optionnels par nature.

Q.

A quoi sert `@RequestBody` ?

Reponse attendue

Il indique a Spring de deserialiser le corps de la requete HTTP (generalement du JSON) directement en objet Java, via Jackson. Combine a `@Valid`, cela declenche aussi la validation Bean Validation sur l'objet deserialise avant l'execution de la methode.

```
1 @PostMapping("/users")
2 public ResponseEntity<UserDto> create(@Valid @RequestBody UserDto dto) {
3     UserDto saved = userService.create(dto);
4     return ResponseEntity.status(HttpStatus.CREATED).body(saved);
5 }
```

Q.

Comment valider les donnees entrantes d'une API REST ?

Reponse attendue

Avec Bean Validation (JSR 380) : on annote les champs du DTO avec `@NotNull`, `@NotBlank`, `@Size`, `@Email`, etc., puis on ajoute `@Valid` devant `@RequestBody` dans le controleur. Si la validation echoue, Spring leve une `MethodArgumentNotValidException`, qu'on intercepte generalement dans un `@ControllerAdvice` pour renvoyer une reponse 400 structuree.

Q.

Comment gerer les exceptions globalement dans une API REST Spring Boot ?

Reponse attendue

Avec `@RestControllerAdvice` (= `@ControllerAdvice` + `@ResponseBody`) combine a des methodes `@ExceptionHandler` : chaque methode intercepte un type d'exception precis et renvoie un code HTTP adapte avec un corps de reponse d'erreur standardise (message, timestamp, code d'erreur), au lieu de laisser fuiter une stack trace ou un code 500 generique.

```
1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3
4     @ExceptionHandler(ResourceNotFoundException.class)
5     public ResponseEntity<ErrorResponse> handleNotFound(
6         ResourceNotFoundException ex) {
7         ErrorResponse body = new ErrorResponse(ex.getMessage(), Instant.
8             now());
9         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
10    }
11
12    @ExceptionHandler(MethodArgumentNotValidException.class)
13    public ResponseEntity<ErrorResponse> handleValidation(
14        MethodArgumentNotValidException ex) {
15        String msg = ex.getBindingResult().getFieldError().
16            getDefaultMessage();
17        return ResponseEntity.badRequest().body(new ErrorResponse(msg,
18            Instant.now()));
19    }
20 }
```

5. Spring Data JPA / Hibernate

Q.

Qu'est-ce que Hibernate et pourquoi utiliser JPA plutot que du JDBC direct ?

Reponse attendue

Hibernate est un *ORM* (Object-Relational Mapping) : il traduit automatiquement les objets Java en lignes de tables SQL et inversement, en generant le SQL a notre place. JPA (Jakarta Persistence API) est la specification standard que Hibernate implemente – coder contre JPA plutot que l'API native Hibernate permet de rester portable entre implementations. Cela evite d'ecrire du JDBC bas niveau repetitif (mapping manuel des `ResultSet`, gestion des connexions).

Q.

A quoi servent `@Entity`, `@Id`, `@GeneratedValue`, `@Table` et `@Column` ?

Reponse attendue

`@Entity` marque une classe comme mappee a une table. `@Id` designe le champ correspondant a la cle primaire. `@GeneratedValue` delegue la generation de cette cle a la base (auto-increment, sequence...). `@Table(name = "...")` et `@Column(name = "...")` permettent de personnaliser le nom de la table/colonne quand il differe du nom de la classe/du champ Java.

Q.

Quelle est la difference entre `CrudRepository`, `PagingAndSortingRepository` et `JpaRepository` ?

Reponse attendue

C'est une hierarchie d'interfaces qui s'enrichissent : `CrudRepository` fournit les operations CRUD de base (`save`, `findById`, `deleteById...`). `PagingAndSortingRepository` l'etend avec la pagination et le tri (`findAll(Pageable)`). `JpaRepository` etend les deux et ajoute des fonctionnalites specifiques a JPA, comme `flush()` (forcer la synchronisation avec la base), `saveAndFlush()`, ou `deleteAllInBatch()` (suppression en une seule requete SQL au lieu d'une par entite).

Q.

Comment fonctionne une methode comme `findByName(String name)` sans aucune implementation ecrite ?

Reponse attendue

Spring Data parse le nom de la methode a la creation du proxy du repository (au demarage) et en deduit automatiquement la requete JPQL correspondante, en se basant sur les noms des champs de l'entite (`findBy` + nom du champ + operateurs comme `And`, `OrderBy`, `GreaterThan...`). Pour des requetes plus complexes, on peut aussi ecrire soi-meme le JPQL avec `@Query`.

Q.

Expliquez les relations `@OneToMany`, `@ManyToOne`, `@ManyToMany` et `@ManyToMany`.

Reponse attendue

Elles mappent les relations SQL classiques : `@ManyToOne` (plusieurs `Order` pour un `Customer`, porte la cle étrangere), `@OneToMany` (le cote inverse, souvent avec `mappedBy`), `@OneToOne` (relation 1-1, ex : `User` et `UserProfile`), `@ManyToMany` (necessite une table de jointure, ex : `Student` et `Course`). Bien identifier le *proprietaire* de la relation (celui qui porte la cle étrangere ou la table de jointure) est essentiel pour eviter des ecritures dupliquees ou incoherentes.

Q.

Quelle est la difference entre `FetchType.LAZY` et `FetchType.EAGER` ? Pourquoi privilegier le Lazy Loading ?

Reponse attendue

`EAGER` charge la relation immediatement avec l'entite parente (requete JOIN ou requete supplementaire systematique). `LAZY` ne charge la relation que lorsqu'elle est effectivement accedee dans le code (via un proxy). On privilegie `LAZY` par default (c'est d'ailleurs le default JPA pour `@OneToMany/@ManyToMany`) pour eviter de charger inutilement de grosses portions du graphe d'objets et degrader les performances – quitte a charger explicitement ce dont on a besoin via `JOIN FETCH` ou `@EntityGraph` quand on sait qu'on va l'utiliser.

Q.

Qu'est-ce que le probleme du "N+1 Select" et comment l'eviter ?

Reponse attendue

C'est un piege classique : Hibernate execute 1 requete pour charger la liste des entites parentes, puis, a cause du Lazy Loading, declenche N requetes supplementaires (une par entite parente) pour charger chacune de leurs collections enfants au moment ou elles sont parcourues – au lieu d'une seule requete avec jointure. On l'evite en utilisant une jointure explicite en JPQL (`JOIN FETCH`) ou en declarant un `@EntityGraph` sur la methode du repository pour forcer le chargement en une seule requete.

```
1 // Provoque potentiellement du N+1 si order.getItems() est LAZY
2 List<Order> orders = orderRepository.findAll();
3 orders.forEach(o -> o.getItems().size()); // 1 requete par order !
4
5 // Solution : jointure explicite en une seule requete
6 @Query("SELECT o FROM Order o JOIN FETCH o.items")
7 List<Order> findAllWithItems();
```

Q.

A quoi sert `CascadeType.ALL` et quels sont les risques a l'utiliser sans reflechir ?

Reponse attendue

Le cascade propage automatiquement une operation (persist, merge, remove...) de l'entite parente vers ses entites liees. **CascadeType.ALL** applique toutes les operations, y compris **REMOVE** : supprimer un parent supprime alors en cascade tous ses enfants. C'est pratique pour une relation de composition forte (ex : **Order** et ses **OrderLine**, qui n'ont pas de sens sans le parent), mais dangereux sur une relation partagee (ex : supprimer un **Author** ne devrait pas supprimer les **Book** s'ils peuvent avoir plusieurs auteurs).

Q.

Comment gerez-vous les transactions dans Spring Boot avec **@Transactional** ?

Reponse attendue

On annote une methode de service (jamais le repository, ni idealement le controleur) avec **@Transactional** : Spring ouvre une transaction avant l'execution et la commit a la fin, ou la rollback automatiquement si une **RuntimeException** (unchecked) non geree est levee – pas par default pour une checked exception, sauf a le preciser avec **rollbackFor**. On peut aussi configurer la **propagation** (ex : **REQUIRED** rejoint une transaction existante ou en cree une, **REQUIRES_NEW** en suspend une existante pour en ouvrir une independante) et l'**isolation** (niveau de visibilite des modifications concurrentes).

Piege classique

@Transactional ne fonctionne que si la methode est appelee *depuis l'exterieur du bean* (Spring l'implemente via un proxy). Un appel interne (**this.autreMethode()**) depuis la meme classe contourne le proxy et ignore silencieusement l'annotation – un piege tres frequemment cite en entretien.

6. SQL

Q.

Quelle est la difference entre `INNER JOIN`, `LEFT JOIN` et `RIGHT JOIN` ?

Reponse attendue

`INNER JOIN` ne retourne que les lignes ayant une correspondance dans les deux tables. `LEFT JOIN` retourne toutes les lignes de la table de gauche, avec `NULL` pour les colonnes de droite si aucune correspondance n'existe. `RIGHT JOIN` fait l'inverse (toutes les lignes de la table de droite).

Q.

Quelle est la difference entre `WHERE` et `HAVING` ?

Reponse attendue

`WHERE` filtre les lignes *avant* l'agregation (`GROUP BY`) et ne peut pas utiliser une fonction d'agregation. `HAVING` filtre les groupes *apres* l'agregation et peut utiliser des fonctions comme `COUNT()`, `SUM()`, `AVG()`.

Q.

A quoi sert un `INDEX` et quel est son cout ?

Reponse attendue

Un index (souvent un arbre B-tree) permet a la base de retrouver des lignes sans parcourir toute la table (*full scan*), accelerant drastiquement les `SELECT` filtres ou tries sur la colonne indexee. Le cout : chaque `INSERT/UPDATE/DELETE` doit aussi mettre a jour l'index, ce qui ralentit les ecritures et consomme de l'espace disque supplementaire – il faut donc indexer les colonnes lues frequemment (cles etrangeres, colonnes de `WHERE/JOIN`), pas tout.

Q.

Quelle est la difference entre `PRIMARY KEY`, `FOREIGN KEY` et `UNIQUE` ?

Reponse attendue

`PRIMARY KEY` identifie de facon unique chaque ligne d'une table (non nulle, unique, une seule par table, souvent indexee automatiquement). `FOREIGN KEY` garantit l'integrite referentielle en imposant qu'une colonne corresponde a une cle primaire d'une autre table. `UNIQUE` garantit simplement l'absence de doublons sur une colonne (ou combinaison de colonnes), sans en faire l'identifiant de la table – et peut accepter `NULL` contrairement a `PRIMARY KEY`.

Exercice SQL – a ecrire soi-meme

On vous donne une table `employees(id, name, department, hire_date, salary)`. Ecrivez les 3 requetes suivantes : (1) les 5 derniers employes embauches, (2) le nombre d'employes par departement, (3) le nom des employes et le nom de leur departement via une jointure avec une table `departments(id, name)`.

*Reflechissez a la clause de tri necessaire pour le (1), a **GROUP BY** pour le (2), et au type de jointure le plus adapte pour le (3) avant de regarder une solution.*

7. Spring Security et JWT

Q.

Quelle est la difference entre authentication et autorisation ?

Reponse attendue

L'**authentication** repond a "qui es-tu ?" (verifier l'identite, ex : login/mot de passe, token).
L'**autorisation** repond a "as-tu le droit de faire ceci ?" (verifier les permissions/roles une fois l'identite etablie, ex : `@PreAuthorize("hasRole('ADMIN')")`).

Q.

Pourquoi utiliser JWT plutot que des sessions serveur classiques ?

Reponse attendue

Un JWT (JSON Web Token) est auto-porteur (*stateless*) : toutes les informations necessaires a l'authentication/autorisation sont encodees et signees dans le token lui-meme, donc le serveur n'a pas besoin de stocker un etat de session. Cela facilite le passage a l'echelle horizontale (pas de partage de session entre instances) et s'integre naturellement dans les architectures microservices ou une API REST est par nature stateless.

Q.

Quelle est la structure d'un JWT ?

Reponse attendue

Un JWT est compose de 3 parties encodees en Base64Url et separees par des points : **Header** (algorithme de signature utilise, ex : HS256), **Payload** (les *claims* : donnees comme l'identifiant utilisateur, les roles, la date d'expiration – non chiffrees, seulement encodees, donc jamais y mettre de secret), et **Signature** (calculee a partir du header, du payload et d'une cle secrete/privee, permettant de verifier que le token n'a pas ete altere).

Piege classique

Le payload d'un JWT n'est pas chiffre, seulement encode en Base64 – n'importe qui peut le decoder et lire son contenu (mais pas le modifier sans invalider la signature). Ne jamais y stocker d'information sensible (mot de passe, numero de carte bancaire).

8. Tests (JUnit, Mockito)

Q.

Comment testez-vous une application Spring Boot a differents niveaux ?

Reponse attendue

Tests unitaires avec JUnit 5 et Mockito : on teste une classe de service isolee, en mockant ses dependances, sans demarrer de contexte Spring (rapide). **Tests d'integration** avec `@SpringBootTest` : on demarre tout ou partie du contexte Spring reel pour verifier que les composants s'assemblent correctement. **Tests de la couche web** avec `@WebMvcTest` : on ne charge que la couche MVC (controleurs, filtres, validation), en mockant la couche service, pour tester le mapping des routes/codes HTTP sans la lenteur d'un contexte complet.

Q.

A quoi servent `@Mock` et `@InjectMocks` avec Mockito ?

Reponse attendue

`@Mock` cree un faux objet dont on controle le comportement (`when(...).thenReturn(...)`) pour simuler une dependance sans son implementation réelle. `@InjectMocks` cree une instance réelle de la classe testee et y injecte automatiquement les champs annotes `@Mock` correspondants – typiquement via l'injection par constructeur si elle existe.

```
1 @ExtendWith(MockitoExtension.class)
2 class OrderServiceTest {
3
4     @Mock
5     private PaymentClient paymentClient;
6
7     @InjectMocks
8     private OrderService orderService;
9
10    @Test
11    void shouldRejectOrderWhenPaymentFails() {
12        when(paymentClient.charge(any())).thenReturn(false);
13
14        assertThrows(PaymentFailedException.class,
15                    () -> orderService.placeOrder(new Order()));
16    }
17 }
```

Q.

A quoi sert `@MockBean` et en quoi differe-t-il de `@Mock` ?

Reponse attendue

`@MockBean` est propre a l'ecosysteme de test Spring : il remplace un vrai bean par un mock dans le contexte d'application Spring lui-meme (utile avec `@SpringBootTest` ou `@WebMvcTest` pour isoler la couche testee des couches sous-jacentes, ex : mocker le **Service** pour tester uniquement le **Controller**). `@Mock` (pur Mockito) cree un mock independant de tout contexte

Spring, utilise dans des tests unitaires purs sans `ApplicationContext`.

9. Maven, Git, Docker

9.1 Maven

Q.

A quoi sert le fichier `pom.xml` ?

Reponse attendue

C'est le descripteur de projet Maven : il declare les coordonnees du projet (`groupId/artifactId/version`), ses dependances, ses plugins de build, et heritages/parents (comme `spring-boot-starter-parent` qui fixe des versions coherentes de dependances).

Q.

Quelle est la difference entre les scopes Maven `compile`, `test` et `runtime` ?

Reponse attendue

`compile` (par default) : la dependance est disponible a la compilation et a l'execution, et se propage aux projets dependants. `test` : disponible uniquement pour compiler/executer les tests (ex : JUnit), absente du jar final. `runtime` : pas necessaire pour compiler le code mais requise a l'execution (ex : un driver JDBC).

9.2 Git

Q.

Quelle est la difference entre `git fetch` et `git pull` ?

Reponse attendue

`git fetch` recupere les commits distants et met a jour les branches de suivi (`origin/main`) sans toucher a la branche locale courante. `git pull` fait un `fetch` suivi automatiquement d'un `merge` (ou `rebase` selon la config) pour integrer ces changements dans la branche locale.

Q.

Quelle est la difference entre `git merge` et `git rebase` ?

Reponse attendue

`merge` cree un nouveau commit de fusion qui combine deux historiques, en preservant fidelement l'historique reel (y compris les branches paralleles). `rebase` rejoue les commits d'une branche par-dessus une autre, produisant un historique lineaire mais reecrivant les hash de commits – a eviter sur une branche deja partagee/poussee sans coordination.

Q.

A quoi servent `git stash` et `git reset` ?

Reponse attendue

`git stash` met de cote temporairement les modifications non commitees (pour changer de branche sans les perdre ni les commiter), qu'on peut restaurer plus tard avec `git stash pop`. `git reset` deplace le pointeur de branche vers un commit anterieur, avec 3 modes : `-soft` (garde les changements indexes), `-mixed` (par defaut, garde les changements mais desindexés), `-hard` (supprime purement et simplement les changements – destructif).

9.3 Docker**Q.**

Pourquoi utiliser Docker pour une application Spring Boot ?

Reponse attendue

Docker encapsule l'application avec toutes ses dependances systeme (JVM, librairies natives) dans une *image* reproductible, garantissant que "ca marche sur ma machine" se traduise par "ca marche partout" (dev, CI, prod). Cela simplifie aussi le deploiement et l'orchestration (Kubernetes, Docker Compose) en traitant chaque service comme une unite isolee et interchangeable.

Q.

Quelle est la difference entre une image et un conteneur Docker ?

Reponse attendue

Une **image** est un template en lecture seule (le "plan de construction", construit a partir d'un **Dockerfile**) contenant le systeme de fichiers et la configuration necessaires pour lancer une application. Un **conteneur** est une instance en cours d'execution de cette image, avec sa propre couche d'ecriture ephemere – on peut lancer plusieurs conteneurs a partir de la meme image.

Q.

A quoi sert un volume Docker ?

Reponse attendue

Par defaut, les donnees ecrites dans un conteneur disparaissent quand il est supprime. Un **volume** est un espace de stockage gere par Docker, monte dans le conteneur, qui persiste independamment du cycle de vie du conteneur – indispensable pour les donnees d'une base de donnees conteneurisee par exemple.

10. Design Patterns

Q.

Expliquez le pattern Singleton et son lien avec Spring.

Reponse attendue

Le Singleton garantit qu'une classe n'a qu'une seule instance, accessible globalement. Spring implemente ce pattern nativement pour le scope par défaut de ses beans : au lieu de coder soi-même un singleton "à la main" (constructeur privé + instance statique, souvent source de bugs en environnement multi-thread ou en test), on délègue cette responsabilité au conteneur, qui garantit une seule instance par contexte applicatif tout en restant facilement testable (on peut recréer un contexte propre par test).

Q.

Expliquez le pattern Factory et où il apparaît dans Spring.

Reponse attendue

Une Factory centralise la logique de création d'objets, évitant d'exposer directement leurs constructeurs et permettant de choisir dynamiquement quelle implémentationinstancier. Spring l'utilise abondamment : l'`ApplicationContext` est lui-même une *BeanFactory* ; les méthodes `@Bean` sont des méthodes-fabriques.

Q.

Expliquez le pattern Builder et donnez un exemple d'usage courant en Java.

Reponse attendue

Le Builder construit un objet complexe étape par étape via des appels chaînés, évitant les constructeurs à rallonge (*telescoping constructor*) et rendant la construction lisible et flexible sur les champs optionnels. Exemple courant : `ResponseEntity.status(201).header(...).body(...)` en Spring, ou l'utilisation de Lombok `@Builder` sur un DTO.

Q.

Expliquez le pattern Strategy et son lien avec l'injection de dépendances.

Reponse attendue

Le Strategy définit une famille d'algorithmes interchangeables derrière une même interface, le choix de l'implémentation concrète étant fait à l'extérieur de la classe qui l'utilise. C'est exactement ce que permet l'injection de dépendances Spring : une classe déclare une dépendance sur une *interface* (ex : `PaymentStrategy`), et le conteneur lui injecte l'implémentation concrète (`CreditCardPayment`, `PaypalPayment`) sans que la classe cliente n'ait à connaître les détails.

Q.

Expliquez le pattern Observer et son lien avec Spring.

Reponse attendue

L'Observer permet a un objet ("sujet") de notifier automatiquement une liste d'"observateurs" abonnées lorsqu'un evenement se produit, sans couplage direct entre eux. Spring l'implemente via son systeme d'evenements applicatifs : `ApplicationEventPublisher.publishEvent(...)` pour emettre, et `@EventListener` pour reagir – utile pour decoupler des actions secondaires (ex : envoyer un email apres une commande) de la logique metier principale.

11. Questions sur votre projet

A retenir Pourquoi c'est 40% de l'entretien

Les recruteurs veulent vérifier que vous *comprenez* vos choix techniques, pas juste que vous savez les nommer. Préparez pour chaque techno de votre stack une réponse en 3 temps : (1) le problème que ça résout, (2) pourquoi cette techno plutôt qu'une alternative évidente, (3) une limite ou un compromis que vous avez accepté.

Q.

Pourquoi avoir choisi Spring Boot pour ce projet ?

Réponse attendue

Exemple de trame de réponse : besoin de mettre en place rapidement une API REST robuste avec un écosystème mature (sécurité, accès aux données, tests) sans reinventer la configuration de base – l'auto-configuration et les starters ont permis de se concentrer sur la logique métier dès le départ plutôt que sur la plomberie.

Q : Pourquoi PostgreSQL plutôt qu'une autre base ?

R : Base relationnelle mature, transactions ACID fiables, bon support JSONB si besoin de flexibilité ponctuelle, écosystème d'outils riche (extensions, monitoring) – à adapter avec les vraies raisons de votre projet.

Q : Pourquoi Kafka dans votre architecture ?

R : Découpler des services pour absorber des pics de charge et permettre un traitement asynchrone/rejouable des événements, plutôt qu'un appel HTTP synchrone fragile entre services.

Q : Pourquoi Docker pour ce projet ?

R : Garantir un environnement reproductible entre les machines de l'équipe, faciliter l'intégration continue, et simplifier l'orchestration de plusieurs services (base, cache, application) avec Docker Compose.

Q : Pourquoi cette architecture (ex : microservices, en couches) ?

R : À justifier par un besoin concret (scalabilité indépendante des composants, isolation des pannes, ou au contraire simplicité pour un petit projet) – éviter de justifier un choix architectural uniquement par "c'est la mode".

Q.

Comment avez-vous sécurisé votre API ?

Réponse attendue

Points à couvrir : authentification (JWT ou session), autorisation par rôles (@PreAuthorize), validation systématique des entrées (@Valid), configuration CORS explicite, gestion des secrets hors du code source (variables d'environnement), et éventuellement rate limiting sur les endpoints sensibles.

Q : Comment gérez-vous les exceptions dans votre projet ?

R : Un @RestControllerAdvice centralise qui traduit chaque type d'exception métier en code HTTP et message standardisés, évitant la duplication de blocs try/catch dans chaque contrôleur.

Q : Comment améliorer les performances de votre application ?

R : Identifier d'abord le goulot d'étranglement réel (profiling, Actuator metrics) avant d'optimiser : pistes courantes = corriger les N+1 queries, ajouter des index cibles, mettre en cache les lectures fréquentes (ex : Spring Cache + Redis), paginer les listes volumineuses.

Q : Que changeriez-vous aujourd'hui dans ce projet ?

R : Réponse honnête et spécifique valorisée par les recruteurs : mieux vaut citer une vraie limite technique assumée (ex : "je testerais plus la couche d'intégration" ou "je découplerais davantage X de Y") qu'une réponse vague du type "rien".

Q : Quel est le plus gros bug que vous avez rencontré et comment l'avez-vous résolu ?

R : Structurez la réponse en méthode : symptôme observé, hypothèses testées, cause racine identifiée, correction, et ce que vous en avez tiré pour éviter la récurrence (test ajouté, monitoring, revue de code).

12. Pieges classiques et questions de reflexion

Q.

Pourquoi Spring est-il basé sur l'IoC plutôt que de laisser le code créer ses propres dépendances ?

Reponse attendue

Sans IoC, chaque classe est fortement couplée aux implementations concrètes de ses dépendances (via `new`), ce qui rend le code difficile à tester (impossible de substituer un mock) et à faire évoluer (changer une implementation implique de modifier tous les appelants). L'IoC inverse cette dépendance : le code dépend d'abstractions, et un composant externe (le conteneur) câble les implementations concrètes – c'est le principe d'inversion des dépendances (le "D" de SOLID) appliqué à grande échelle.

Q : Pourquoi programmer contre des interfaces plutôt que des implementations concrètes ?

R : Cela permet de changer d'implementation sans toucher au code appelant, et de substituer facilement un mock en test – c'est la base du principe de substitution de Liskov et de l'injection de dépendances.

Q : Pourquoi REST plutôt que SOAP ?

R : REST s'appuie sur les verbes et codes HTTP standards, est plus léger (souvent JSON vs XML enveloppe SOAP), plus simple à consommer et à faire évoluer, et s'intègre naturellement au cache HTTP – SOAP reste pertinent pour des contrats stricts (WSDL) et des besoins avancés (transactions distribuées, WS-Security).

Q : Pourquoi dit-on que HTTP est stateless et quel impact sur la conception d'une API ?

R : Chaque requête HTTP est indépendante et ne s'appuie sur aucun contexte serveur laissé par une requête précédente – cela pousse à authentifier chaque requête individuellement (ex : token JWT dans le header) plutôt que de compter sur une session serveur, ce qui facilite la scalabilité horizontale.

Q : Pourquoi le pattern Singleton est-il à la fois utile et potentiellement dangereux ?

R : Utile pour partager un état/service coûteux à instancier ; dangereux si l'état qu'il contient est mutable et partagé entre threads sans synchronisation, ou si son usage rend le code difficile à tester (dépendance globale cachée) – Spring atténue ce risque en gérant le cycle de vie et en facilitant l'injection de mocks en test.

Q : Pourquoi `@Transactional` peut échouer silencieusement lors d'un appel interne ?

R : Spring implémente les annotations comme `@Transactional` via un proxy généré autour du bean. Un appel via `this.` depuis l'intérieur de la même classe contourne ce proxy et exécute directement la méthode originale, sans jamais passer par la logique transactionnelle – d'où l'importance d'extraire la méthode annotée dans un autre bean si nécessaire.

13. Exercices de code en direct

A retenir Methode conseillée en live coding

Avant d'écrire la moindre ligne : reformulez le besoin a voix haute, identifiez les entites/DTOs necessaires, puis annoncez votre plan (endpoint -> service -> repository) avant de coder. Cela montre une demarche structuree meme si le code final n'est pas parfait.

CRUD complet pour une entite Product

Implementez un CRUD REST complet pour une entite **Product** (id, name, price, stock) : entite JPA, repository Spring Data, service, controleur avec les 5 operations standard (create, findById, findAll, update, delete) et les codes HTTP appropriés pour chacune.

Points d'attention attendus par l'évaluateur : gestion du cas "produit non trouve" (404), utilisation de DTOs plutot que d'exposer l'entite JPA directement, et validation des champs a la creation.

Endpoint de connexion avec JWT

Implementez un endpoint **POST /auth/login** qui recoit un couple email/mot de passe, verifie les identifiants, puis retourne un JWT signe contenant l'identifiant utilisateur et son role, avec une date d'expiration.

Reflechissez a : ou stocker la cle de signature (jamais en dur dans le code source), comment hacher le mot de passe stocke (BCrypt), et quel code HTTP renvoyer si les identifiants sont invalides.

Pagination et recherche

Ajoutez a un endpoint **GET /products** la possibilite de paginer les resultats (page, size) et de filtrer par mot-cle sur le nom du produit, en utilisant **Pageable** et **JpaRepository**.

Reflechissez a la methode de repository a ecrire (nom de methode derive vs @Query) et a la structure de la reponse JSON paginee (contenu + metadonnees de pagination).